

Backend Technology Stack

Costimiser AI Analytic Engine Concise description of the application, analytics, RAG, cloud, and deployment stack

Layer	Technologies
Application	Python 3.11, Flask 3.0.3
Analytics	pandas, NumPy, SciPy, scikit-learn, SHAP
RAG	Amazon Bedrock, Amazon Titan Text Embeddings V2, FAISS CPU, Anthropic Claude Sonnet 4.5
Output	PyArrow, Fastparquet, Plotly
AWS	boto3, s3fs, Amazon S3, AWS STS, Systems Manager Parameter Store, IAM Roles for Service Accounts
Deployment	Docker, Amazon EKS, Helm 3
Infrastructure and delivery	Terraform 1.15.8, AWS CodeBuild

Application runtime

The backend is implemented in **Python 3.11** and exposes its endpoints through **Flask 3.0.3**. The application is packaged as a Docker container and deployed to Amazon EKS.

Runtime consideration

The current container starts the application with Flask's built-in server. For higher concurrency and stronger worker management, a dedicated WSGI server such as Gunicorn could be considered.

Application architecture

The backend follows a layered structure with separate modules for API endpoints, orchestration, services, tools, configuration, and shared utilities. The `/ask-card` endpoint interprets natural-language requests, selects the appropriate analytical card, and delegates execution to the corresponding tool. Each tool contains the main analytical logic and manages access to its required dependencies.

Data and analytical processing

The main data and numerical libraries are **pandas 2.3.3**, **NumPy 2.3.3**, and **SciPy**. They support process-data preparation, aggregation, statistical computation, scenario analysis, and optimization.

Machine learning and explainability

The backend uses **scikit-learn 1.7.2** for prediction models and analytical pipelines, and **SHAP** for model explainability. These capabilities support prediction, diagnosis, scenario evaluation, recommendations, and optimization.

Retrieval-Augmented Generation

The backend includes a RAG subsystem for papermaking knowledge retrieval and for enriching analytical recommendations. It uses **Amazon Bedrock**, **Amazon Titan Text Embeddings V2**, **FAISS CPU**, and **Anthropic Claude Sonnet 4.5**. Titan converts document chunks and user questions into embeddings, FAISS retrieves the most relevant papermaking content, and Claude generates the grounded answer or recommendation.

Data serialization and analytical output

The backend uses **PyArrow**, **Fastparquet**, and **Plotly 5.24.1** to support tabular data exchange, downloadable analytical results, and interactive figures.

AWS integration

The service uses **boto3**, **s3fs**, **Amazon S3**, **AWS STS**, **AWS Systems Manager Parameter Store**, and **IAM Roles for Service Accounts**. These services support data and model access, artifact handling, secure configuration, and AWS

permissions.

MLflow status

MLflow integration is planned but is not yet part of the operational backend stack.

Containerization and deployment

The application is packaged with **Docker** and deployed to **Amazon EKS**. The Kubernetes resources are generated using **Helm 3**.

Infrastructure as code

The backend infrastructure is managed with **Terraform 1.15.8**.

CI/CD

The build and deployment process runs in **AWS CodeBuild**. The pipeline performs application validation, container build, security checks, infrastructure deployment, and rollout to EKS.

Security and quality controls

The delivery process includes controls for static code analysis, Python style validation, dependency and container vulnerability scanning, Kubernetes manifest validation, code coverage, XML test reporting, non-root container execution, restricted container privileges, read-only container filesystems, and IAM-based access control.

Summary

The backend is a Python 3.11 Flask analytical service that combines data processing, machine-learning inference, explainability, optimization, and Retrieval-Augmented Generation. The RAG subsystem uses Amazon Bedrock with Amazon Titan Text Embeddings V2, FAISS CPU, and Anthropic Claude Sonnet 4.5. The service is containerized with Docker, deployed to Amazon EKS using Helm, managed with Terraform, and delivered through AWS CodeBuild.